
Cours 1 – 4IN010

Introduction au Noyau

Pierre.Sens@lip6.fr

Pré-requis et objectifs

Pré-requis : 1 cours de principes de systèmes en L, langage C

Objectifs : Etude des mécanismes internes du noyau (UNIX)

- Processus,
- Ordonnancement
- Fichiers,
- Mémoire virtuelle

Organisation de l'UE

Début TD : La semaine du 18 septembre

Notation : 40 % Examen réparti 1, 60 % Examen réparti 2

Equipe enseignante

Cours : Pierre Sens

TD : Luciana Arantes, Pierre Sens, Julien Sopena

{Prénom.Nom}@lip6.fr

TD: Etude noyau Unix V6 (~10 000 lignes de code C)

moodle : accès source Unix

Bibliographie

Mécanismes internes du noyau UNIX

- M.J. Bach, «Conception du système UNIX»,
- S.J Leffler & al. , «Conception et implémentation du système 4.4 BSD»,
- U. Vahalia, «Unix internals --the new frontiers»

Noyau Linux :

- D. P. Bovet, M. Cesari, « Le noyau Linux »
- R. Love, « Linux Kernel Development »

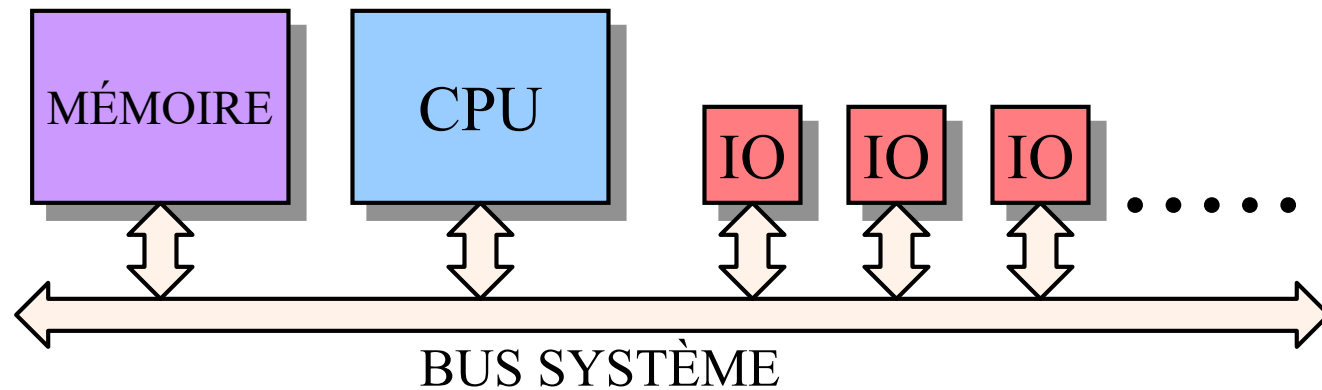
Mécanismes internes Windows

- Windows® Internals

Vue d'une machine

Ensembles de composants organisées autour de **bus**

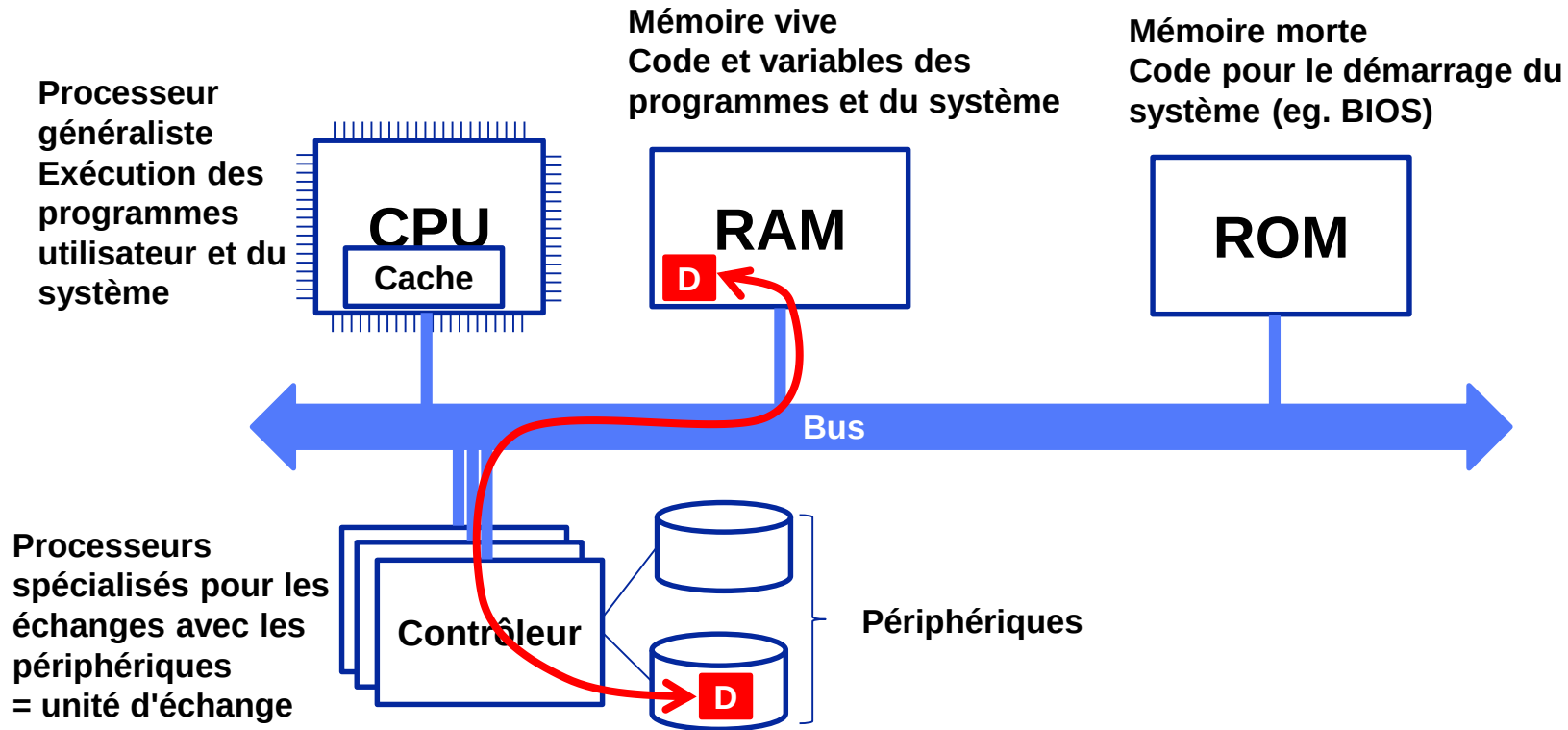
Composants de base: mémoire, CPU, I/O, bus système



I/Os “standard”: disques (cartes SCSI, IDE, SATA ...), clavier, souris, haut-parleurs, etc.

La vitesse du bus système devient le facteur prédominant pour la performance d'un ordinateur.

Composants d'un ordinateur

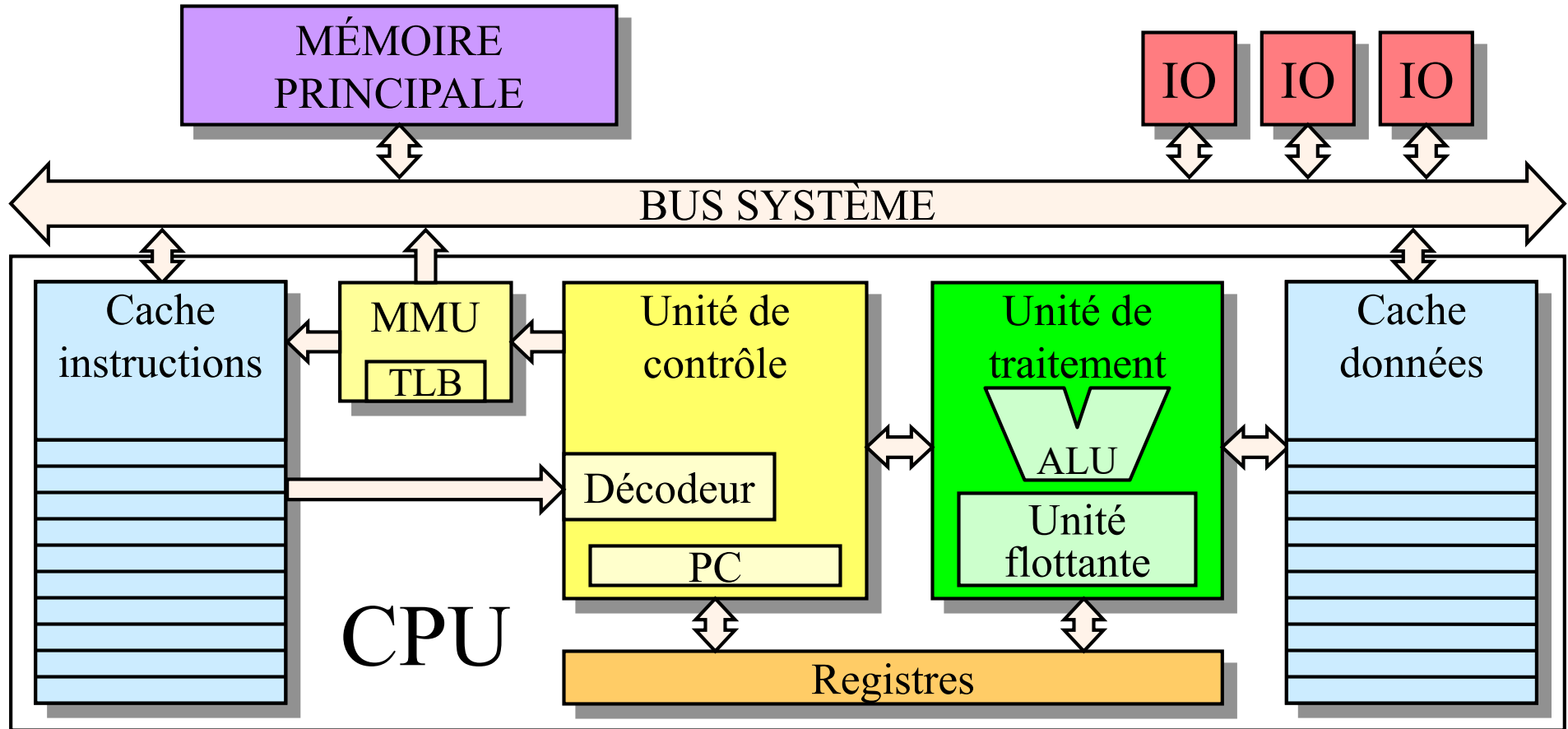


1 contrôleur par type de périphérique (disque, réseau,)

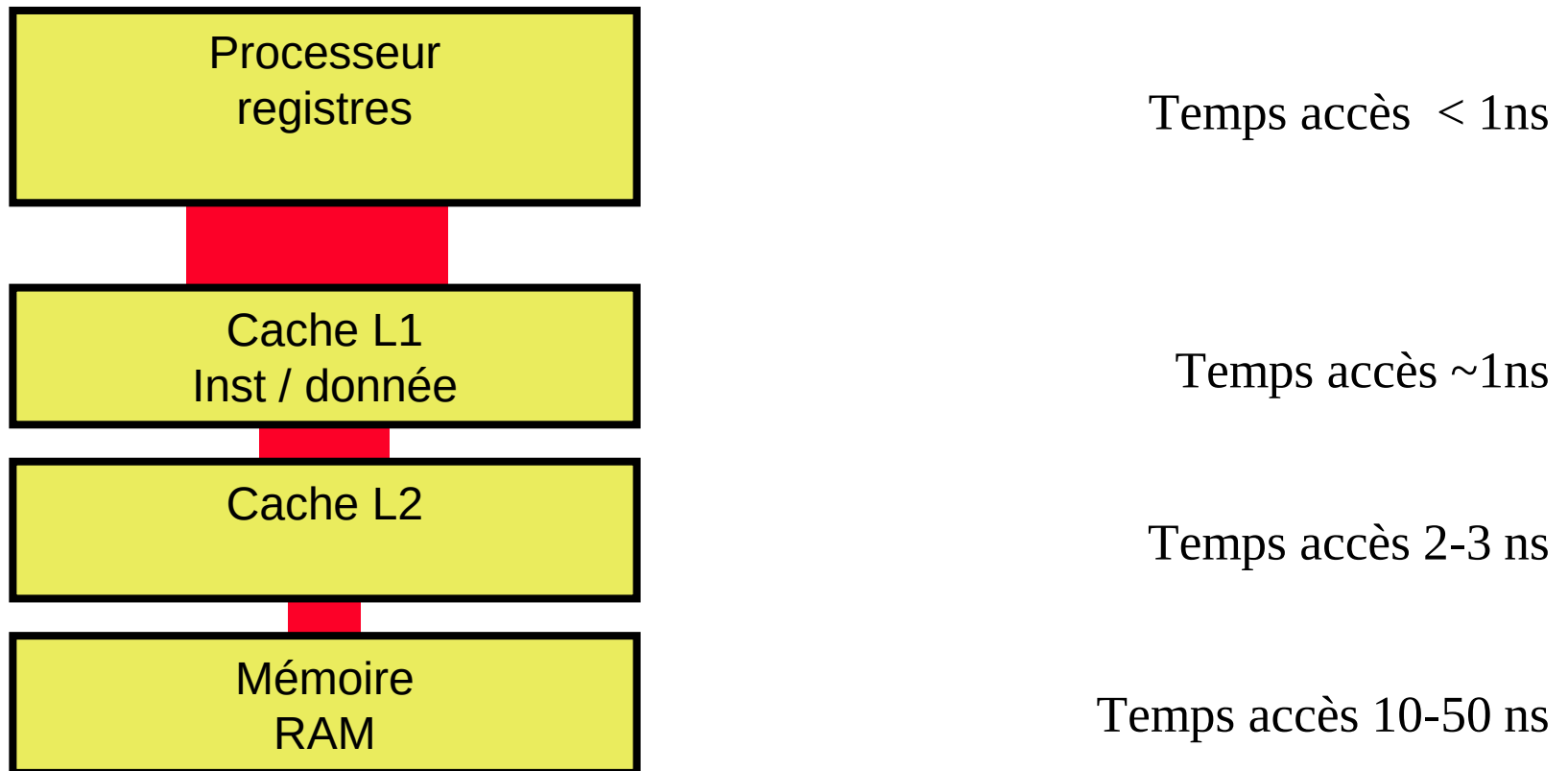
Accès direct à la mémoire : **DMA** (Direct Memory Access)

Entrée/sortie (E/S) : entrée = lecture, sortie = écriture

Vue d'une machine : le processeur central

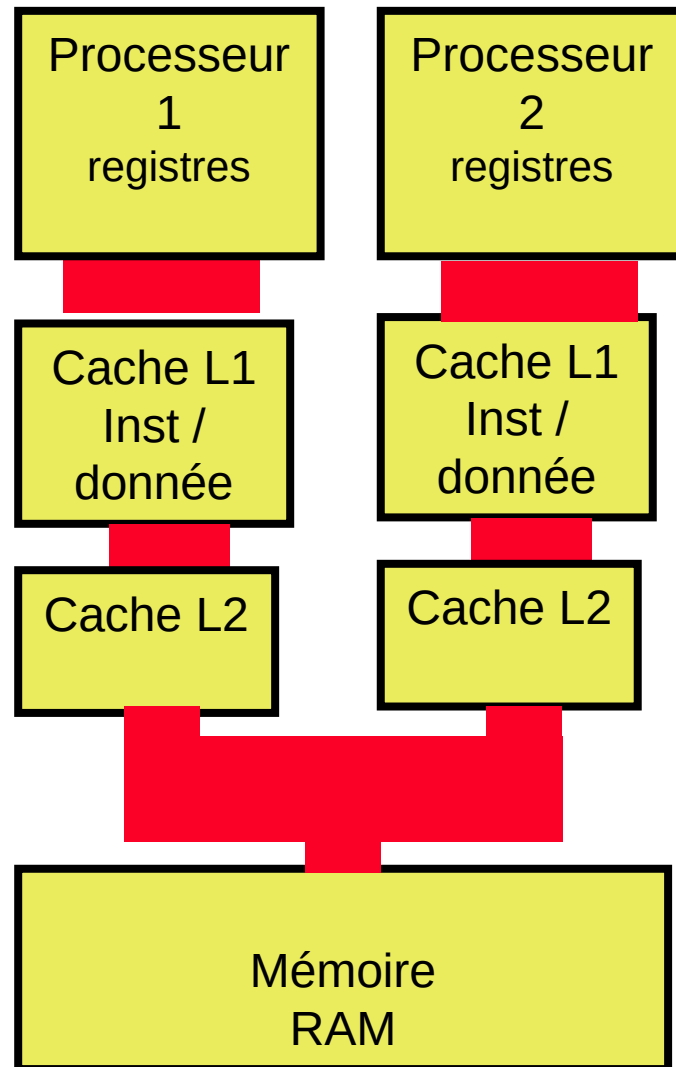


Vue d'une machine : Les niveaux de caches



Disque : 10 ms ($\times 10^6$ RAM !)

Vue d'une machine : Multi-processeur SMP



Architecture Symmetric Multiprocessor (SMP)

- **Processeurs distincts**, pas de partage de caches

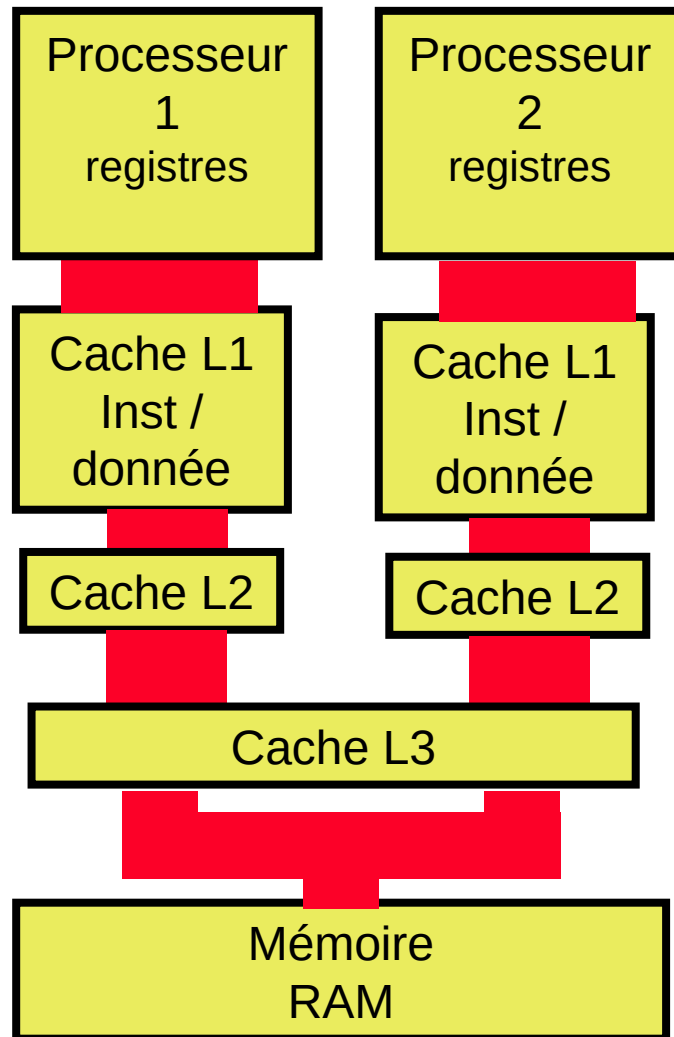
- **Partage de la RAM**

- + Gestion de flux indépendants

- Maintien de la cohérence des caches

- Conflit d'accès au bus mémoire (nb de processeurs limités)

Vue d'une machine : Multi-core



Architecture *Multi-core*

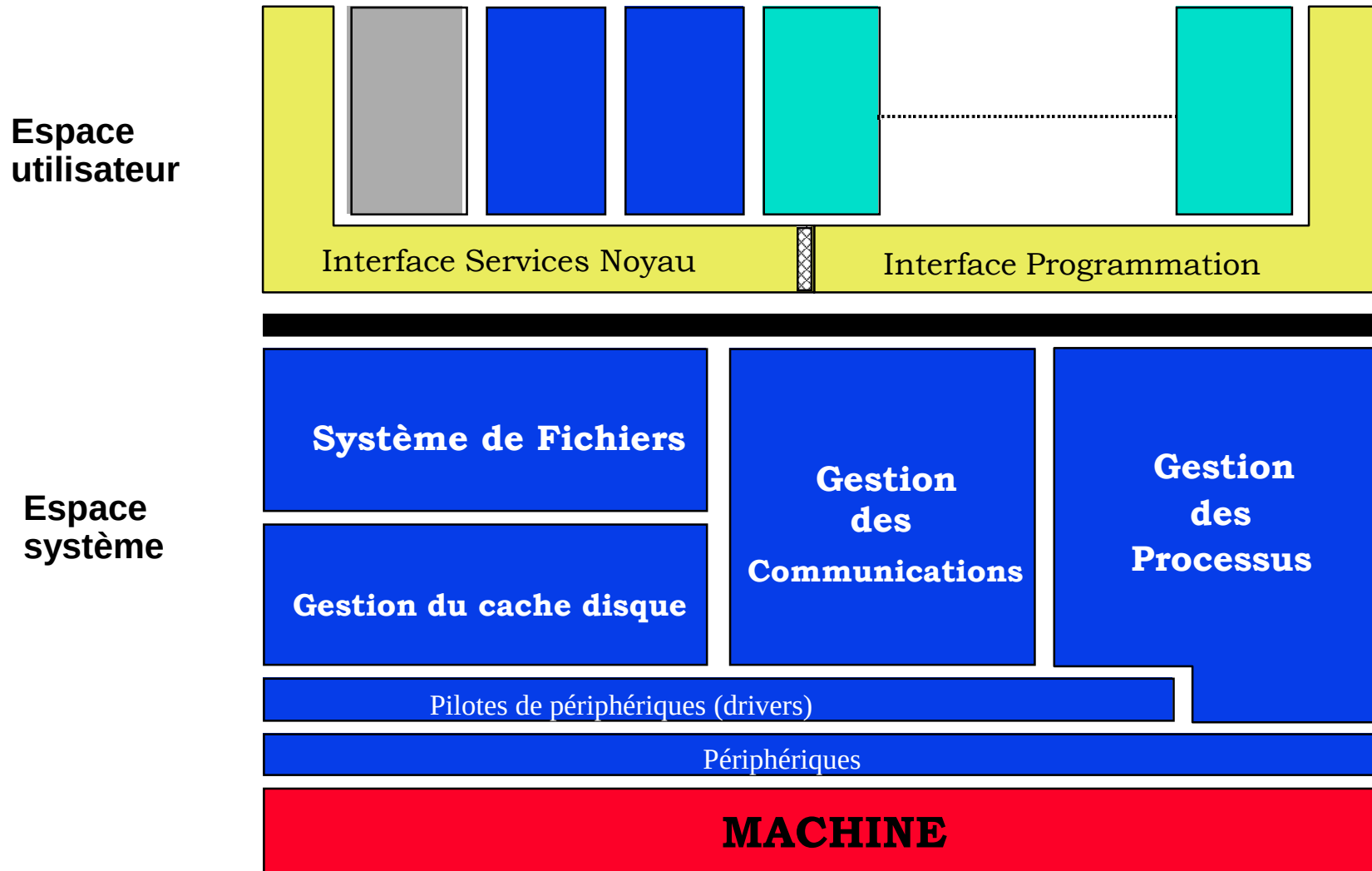
(ex. Intel, AMD)

- Deux cœurs de processeurs distincts sur un même support, **partage de caches** possible (L2 ou L3)

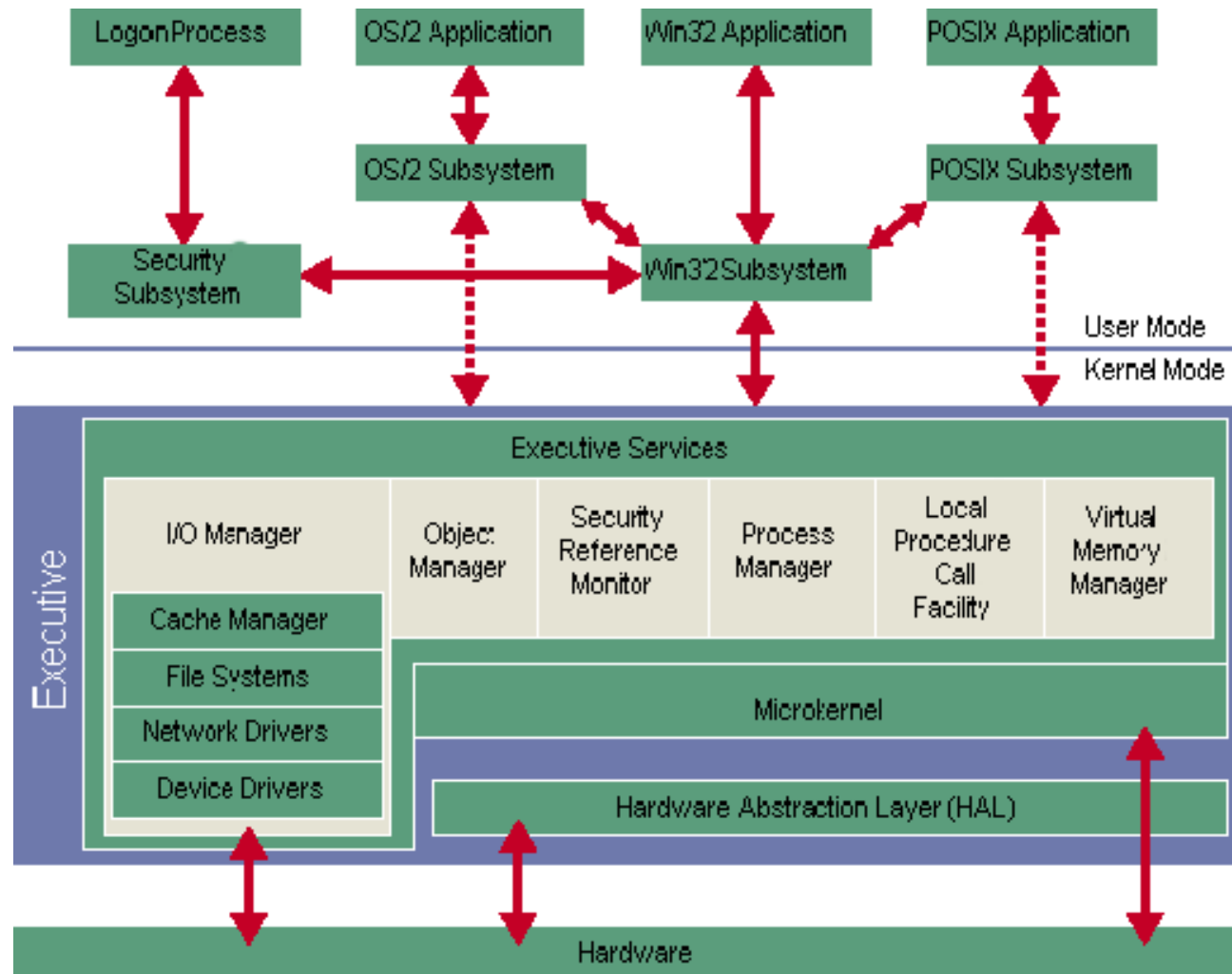
- **Partage de la RAM**

- + Gestion de flux indépendants
- + Moins coûteux que SMP
- + Moins de pb de cohérence
- Moins de cache disponible

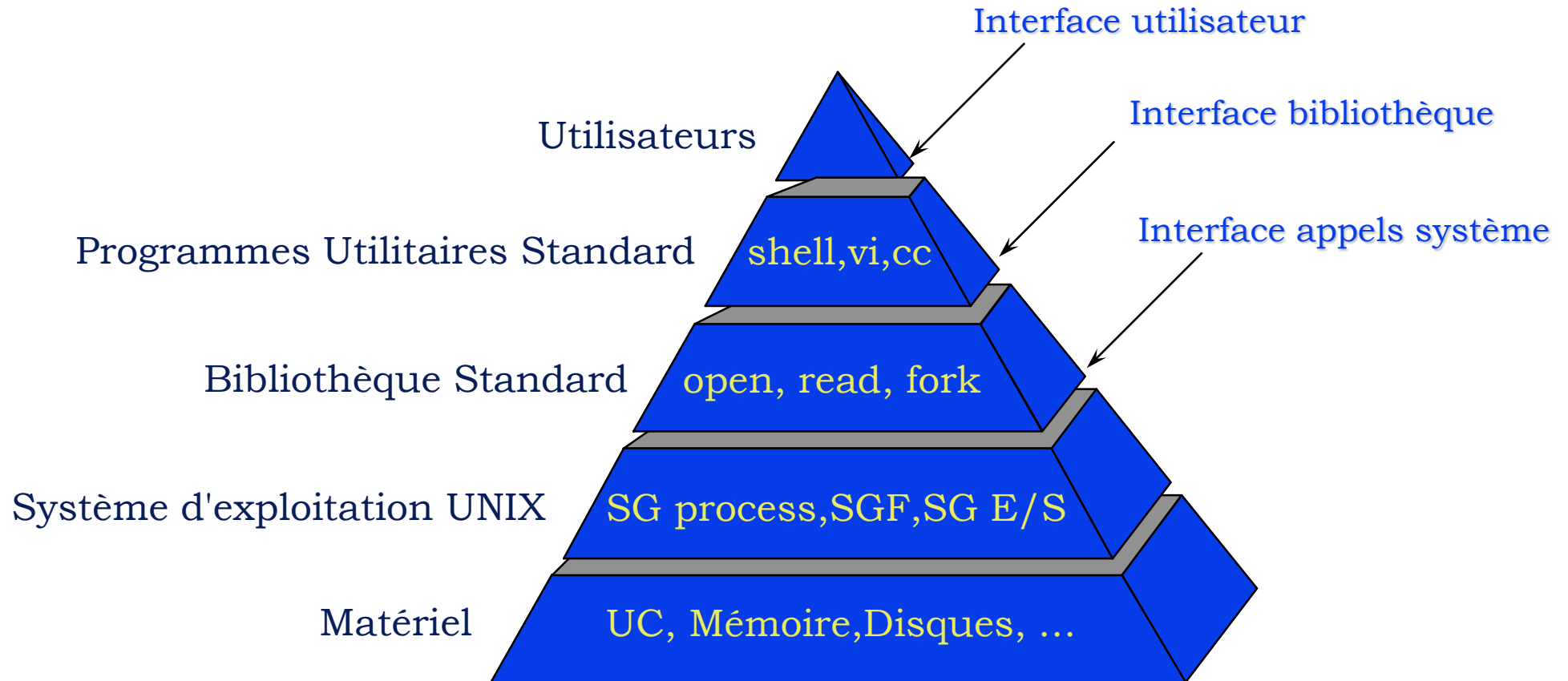
Vue du système : architectures classiques



Vue du système : architectures modulaire (Windows NT)



UNIX - Généralités (1)



UNIX - Généralités (2)

Système interactif en temps partagé, Multi-Utilisateurs et Multi-Tâches

Principes

- Système de gestion de fichiers hiérarchisé

- Entrées/Sorties

- Création dynamique de processus (Père / Fils)

- Communication inter-processus (Pipes, Sockets)

Langage de commande extensible (Shell)

Noyau monolithique portable

- Le noyau est écrit en C à 95%.

- UNIX existe sur de nombreuses machines (PC, Stations RISC, CRAY-YMP, Hypercubes, ...)

Historique Unix

A l'origine UNICS (UNIplexed information and Computing System)

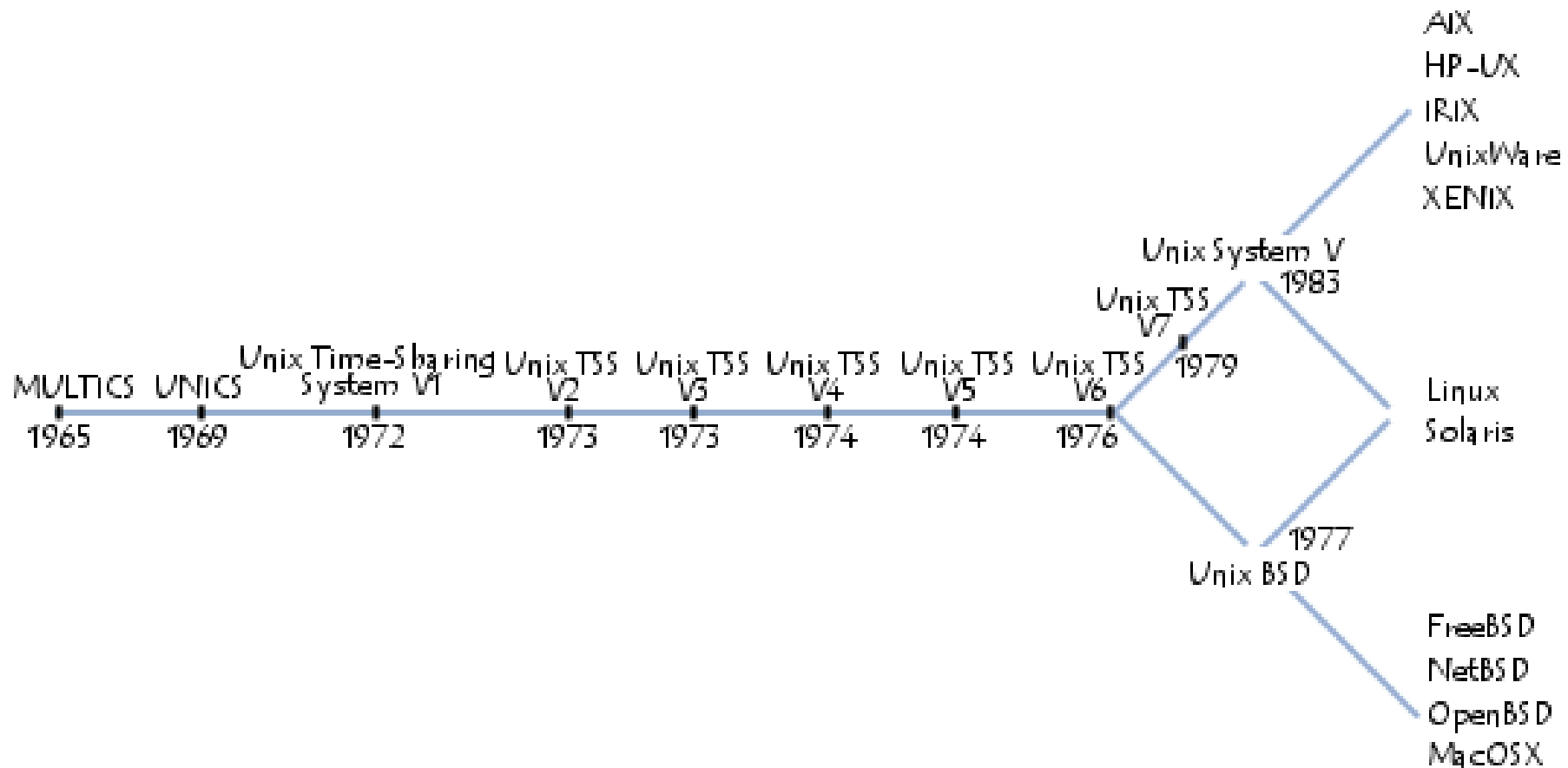
1969 UNIX/PDP-7 Ecrit en assembleur sur un PDP-7 pour développer un traitement de texte aux Bell Labs

1973 UNIX V5 Langage C (90%)

1976 UNIX V 6 (coopération de Bell Labs/universités américaines)

	Berkeley	Bell Labs	AT&T
1977	1.0 BSD	-	-
1978	2.0 BSD	V7	-
1979	3.0 BSD	-	-
1980	4.0 BSD	-	-
1981	4.1 BSD	-	-
1982	-	-	System III
1984	4.2 BSD	V8	System VR2
1986	4.3 BSD	-	-
1989	-	V 10	-
1993	4.4 BSD	-	System VR4
1991		Linux	

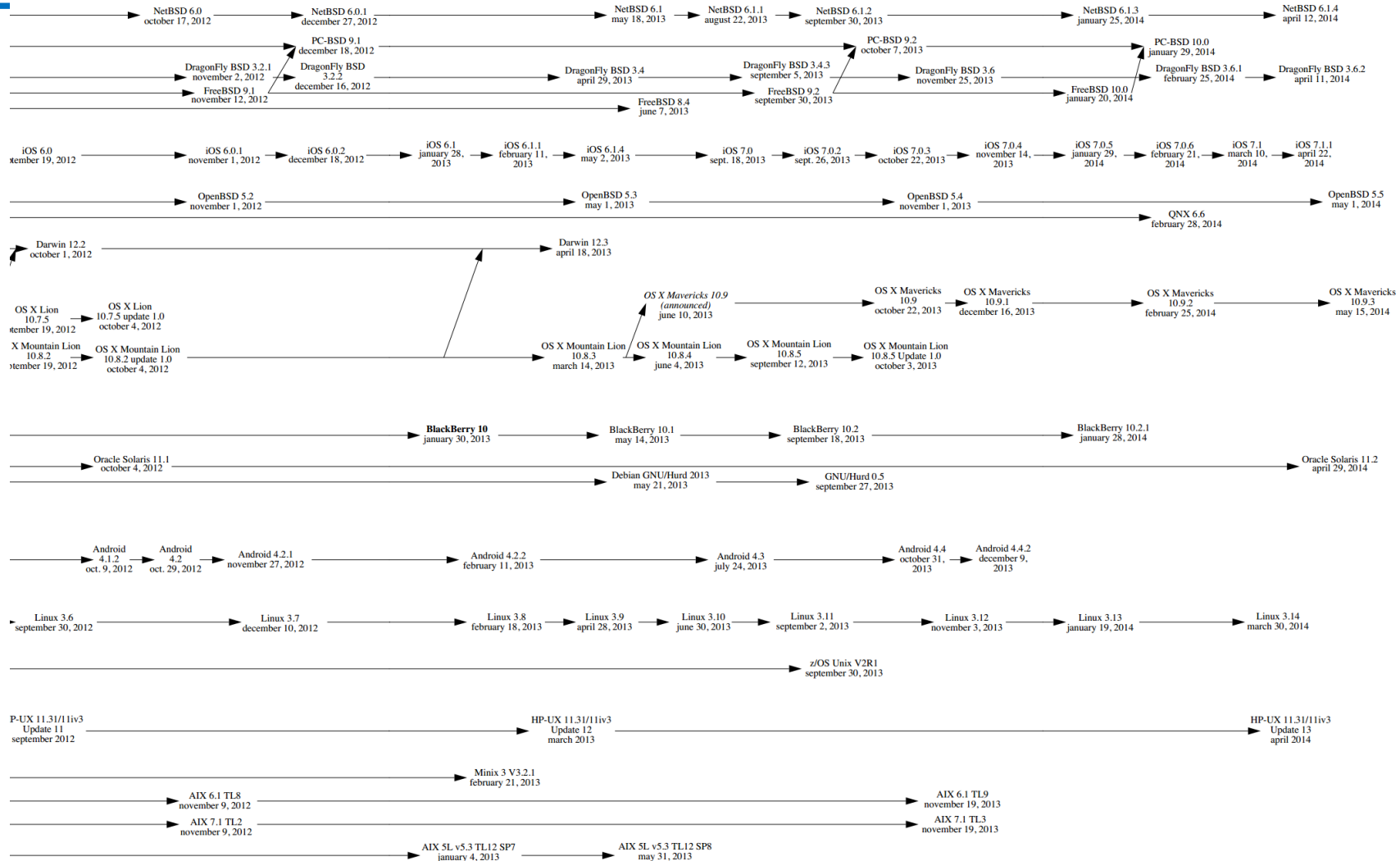
Historique Unix



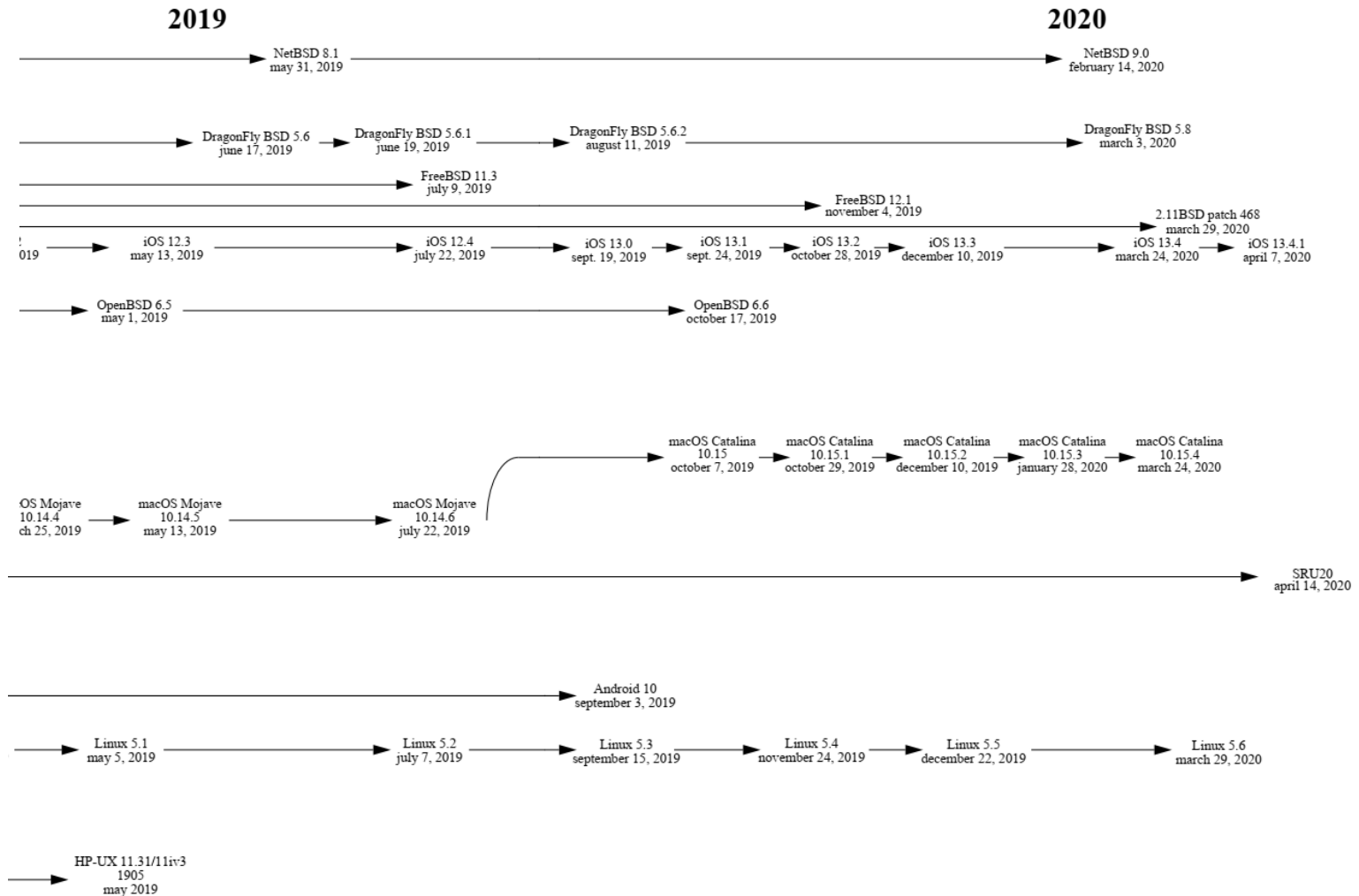
Historique Unix

2013

2014



Unix aujourd'hui



Mécanismes Internes Unix

Architecture générale

Gestion de processus

Ordonnancement/ Signaux / Multi-thread

Gestion de fichiers

SGF / Entrées-sorties disque

Gestion mémoire

Mémoire virtuelle

Notion de processus

Processus : exécution des applications

Associé à un espace isolé en mémoire = **espace d'adressage**

Code

Pile d'exécution : variables locales, sauvegarde de contexte appel de fonction

Données : variables globales, tas (allocation dynamique)

Contexte matériel du processus

Ensemble de registres

Prochaine instruction : PC (Program Counter) / CO (Compteur Ordinal) / IP
(Instruction Pointer – Intel)

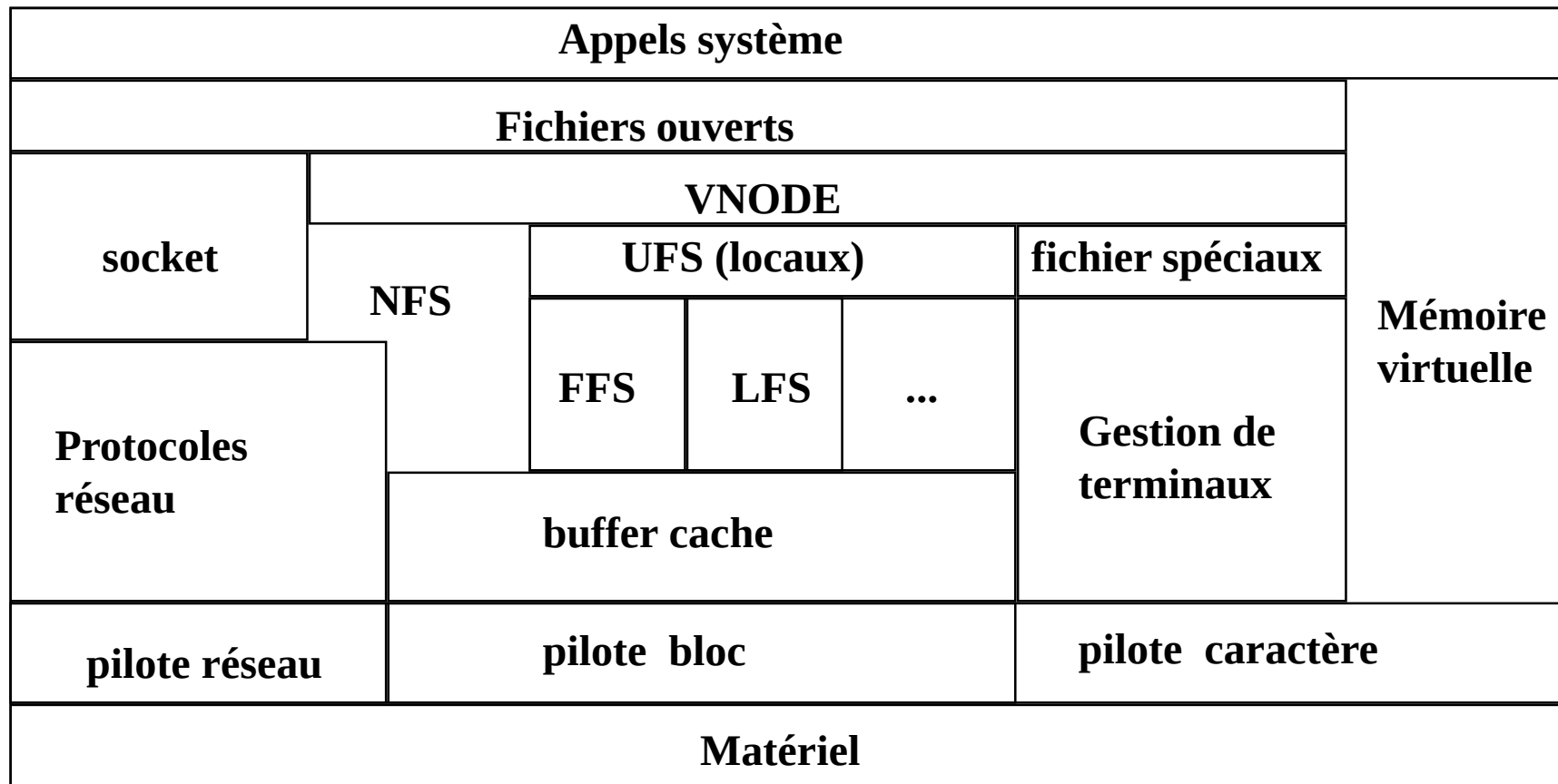
Registre de pile (SP) ...

Architecture générale

Processus 1

Processus 2

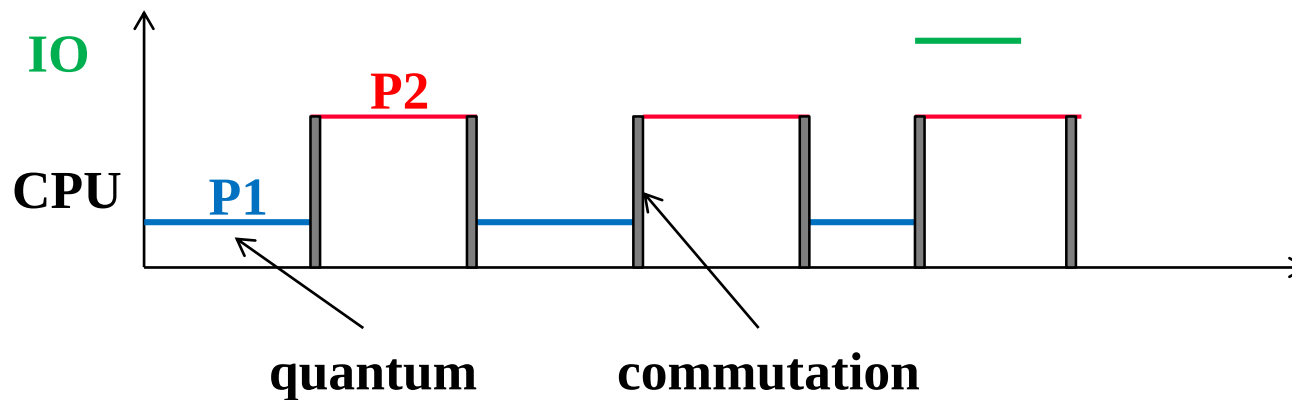
Processus 3



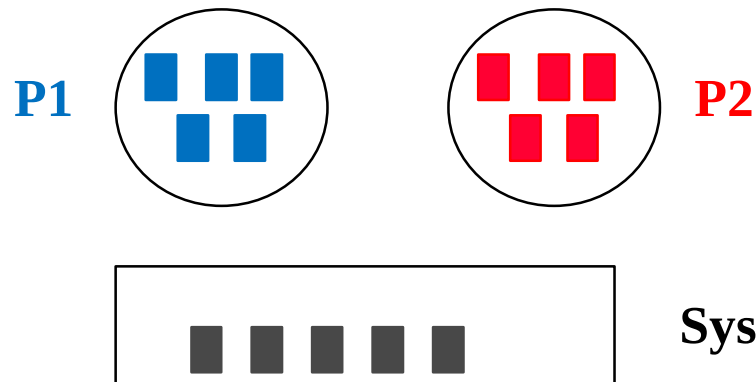
Rôles du système d'exploitation

Partager les ressources entre les processus

1) Processeur : Ordonnancement (scheduling)



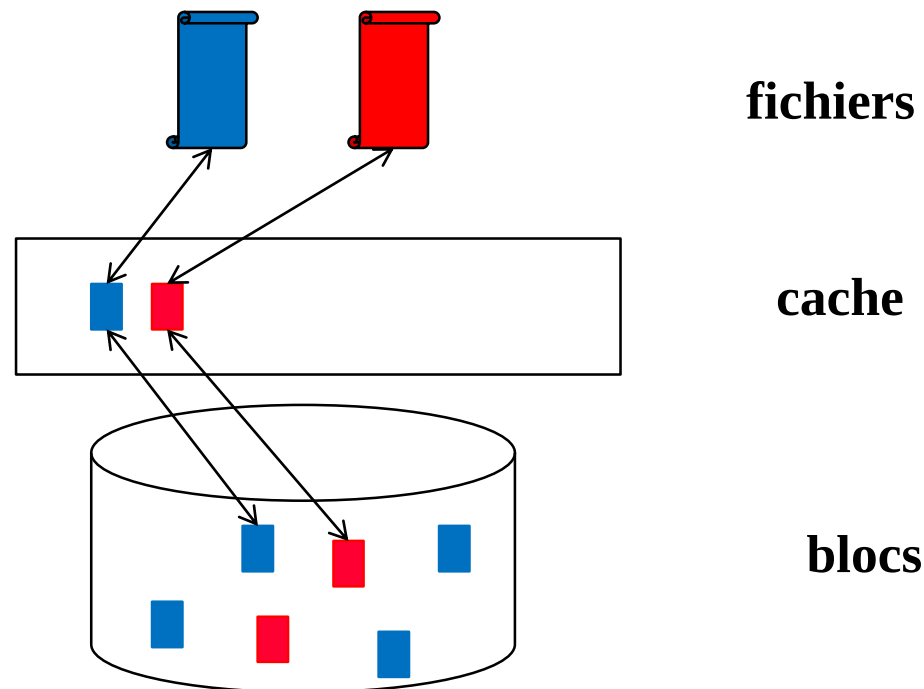
2) Mémoire : assurer l'isolation et le partage (mémoire virtuelle)



Rôles du système d'exploitation

Partager les ressources entre les processus

3) Disque : Système de fichiers

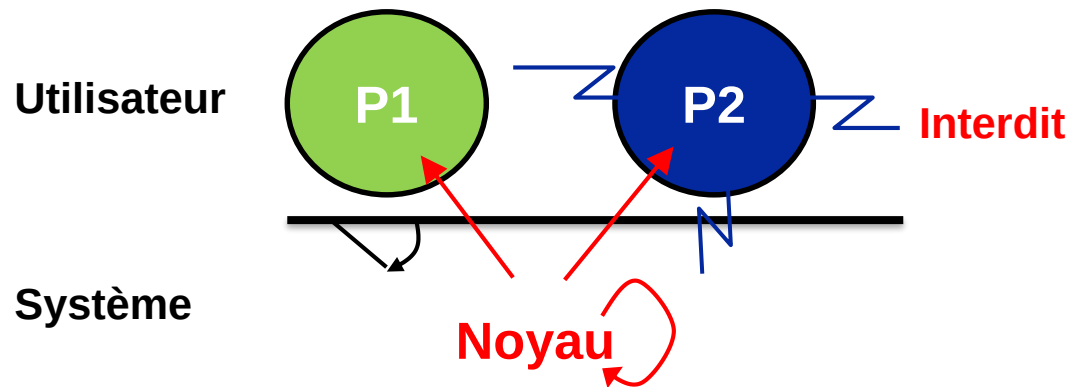


Modes d'exécution

2 modes

Mode utilisateur : accès restreint (confinement) aux ressources
Exécution des codes des programmes

Mode système : accès privilégié à toutes les ressources du système
Exécution des codes du noyau.



Accès aux services du noyau

Appels système (interruption logicielle - trappe)

A l'initiative du processus courant (synchrone)

Utilisation de la **pile système** du processus courant

Interruptions matérielles – événements *externes*

Evénements externes asynchrones indépendants du processus courant (exemple : périphériques d'E/S)

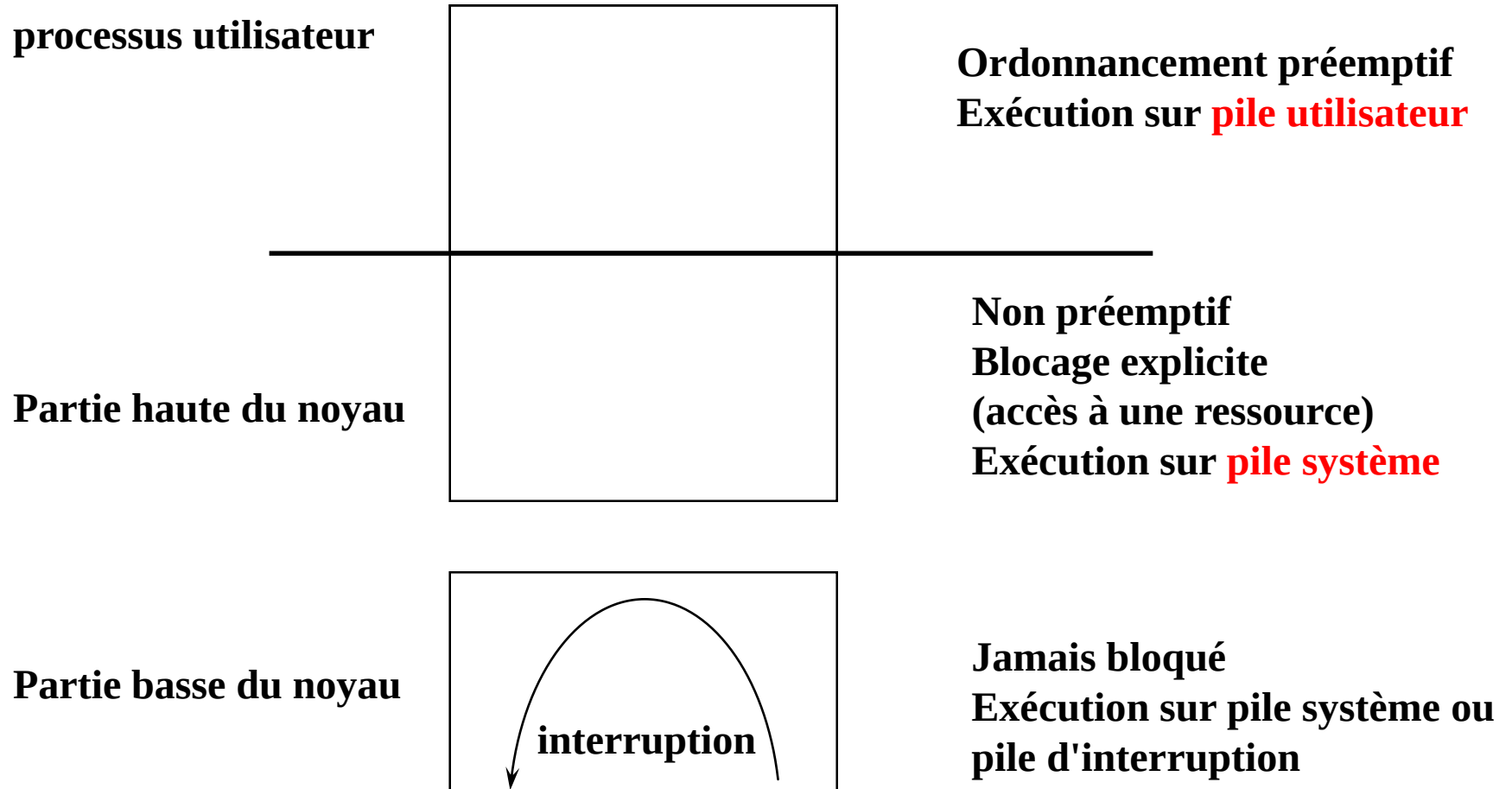
Utilisation de la **pile d'interruption ou pile système**

Interruptions matérielles – événement *internes*

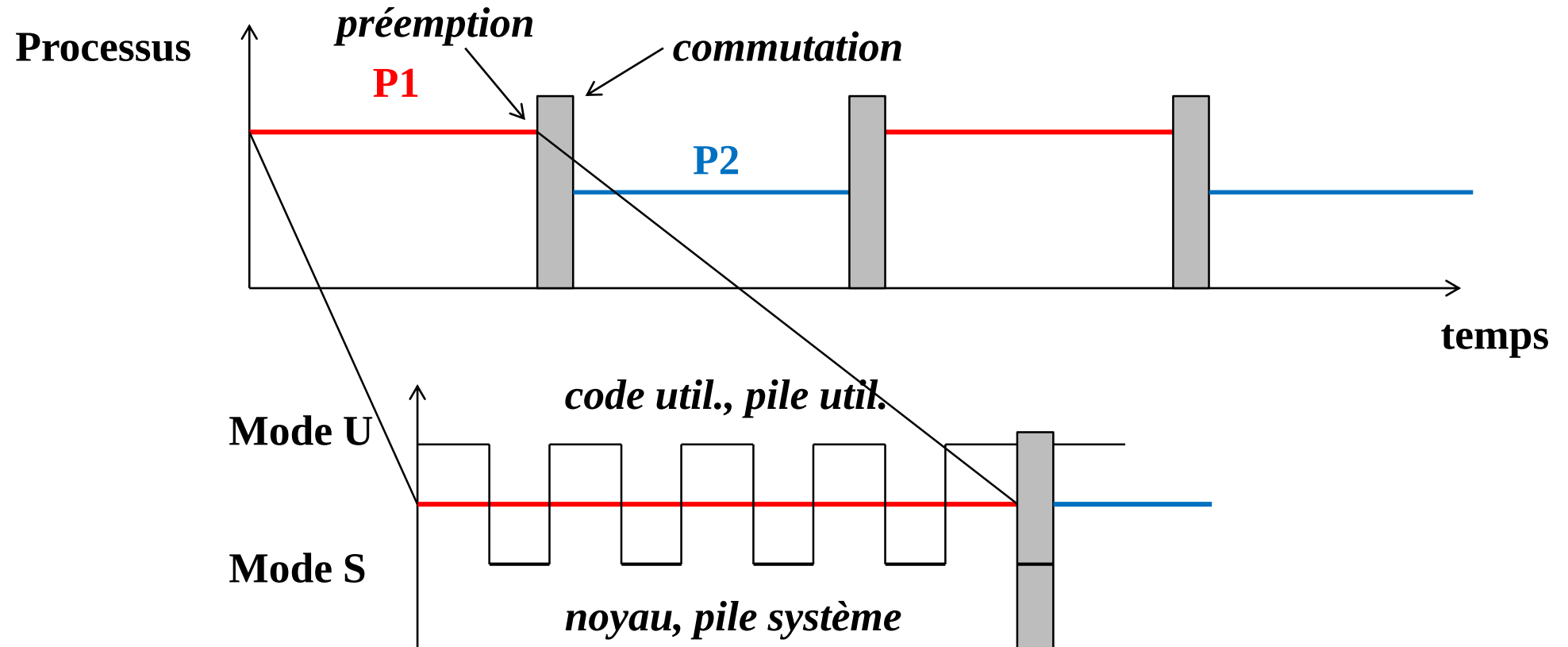
Evénements liés au processus courant (ex : division par zéro, erreur d'adresse)

Utilisation de la **pile système**

Organisation et mode d'exécution



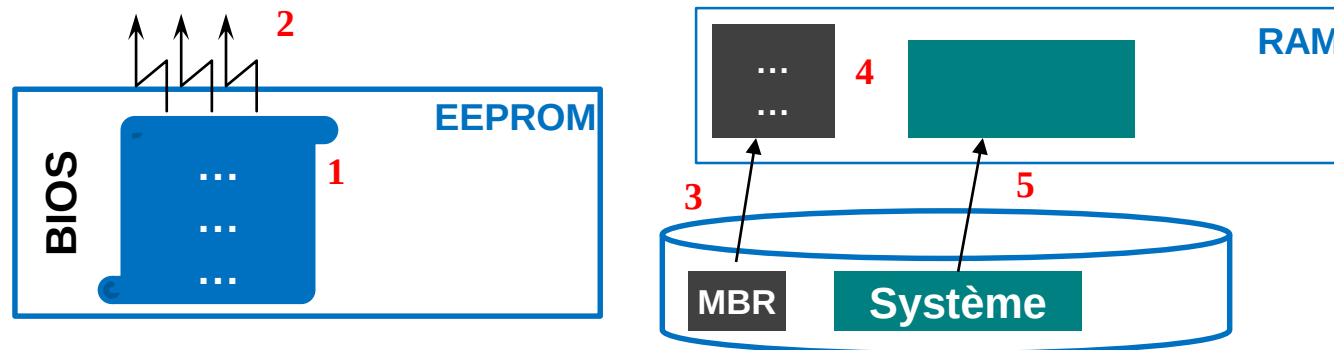
Ordonnancement et mode d'exécution



Amorçage du système (boot)

A la mise sous tension la RAM est vide : charger et initialiser le système en RAM

⇒ Utilisation d'un "mini" OS présent en EEPROM (le BIOS sur PC)

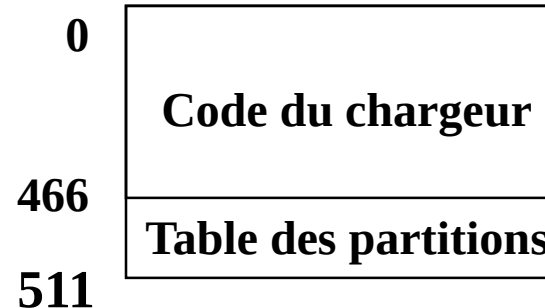


Séquence d'amorçage

1. Mise sous tension : exécuter le code présent en EEPROM (BIOS)
 2. Vérifier les périphériques
 3. Copier le contenu du premier bloc du disque en RAM (MBR – Boot loader Block)
 4. Exécuter le boot loader en RAM
 5. Charger le système d'exploitation
- BIOS
- Loader

boot bloc et multi-boot

Format :



Multi-boot (grub, lilo, boot manager) :

Changer le code du chargeur du boot bloc de la première partition (MBR)

Ex: lilo

- 1) Exécution chargeur lilo (inclus dans bloc 0 du premier disque)
lilo:
- 2) Choix de la partition à booter
- 3) Lire la table des partition pour trouver le boot bloc de la partition
- 4) Exécuter le chargeur du boot bloc trouvé

Bios vs EFI

Limitation du BIOS

Mode 16 bits

Utilisation du MBR => taille de partition limitée :

Entrée de 32 bits :

Taille max = 2^{32} blocs = 2 To

=> EFI (Extensible Firmware Interface)

1 partition pour stocker les informations de boot **GUID Partition Table (GPT)**

GPT contient adresse programme d'armorçage et la table des partitions

Entrée sur 64 bits => Taille max = 2^{64} blocs

Les principales structures

